Propositional Logic Evaluator

by

Aye Khin Khin Hpone (Yolanda Lim)

Student ID: st125970

st125970@ait.asia

AT70.07 — Programming Languages and Compilers

Assignment 1

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akraradet Sinsamersuk

Asian Institute of Technology

School of Engineering and Technology

Thailand

March 2026

ABSTRACT

This report presents the design and implementation of a Propositional Logic Evaluator using classical compiler front-end techniques. The system accepts propositional logic expressions composed of truth values (`t`, `f`) and binary connectives ($\wedge$, $\vee$), and produces two outputs: the evaluated truth value and an equivalent expression in prefix (Polish) notation. The implementation follows a three-phase pipeline — lexical analysis, LALR(1) parsing with syntax-directed AST construction, and AST-based evaluation and translation. A context-free grammar with declarative precedence and associativity rules resolves ambiguity, and the SLY parser-generator library ensures a direct correspondence between the formal grammar and the Python implementation. An interactive GUI built with PySide6 (Qt6) provides convenient expression input and result display. All six required test cases pass correctly.

CONTENTS

ABSTRACT **ii**

1 INTRODUCTION AND LANGUAGE DESIGN **1**

1.1 Background and Problem Statement 1

1.2 Propositional Logic 1

1.3 Alphabet and Token Specification 2

1.4 Context-Free Grammar 2

1.4.1 Equivalent Unambiguous Grammar 3

1.5 Operator Precedence and Associativity 4

1.6 Notation Systems 4

2 SYSTEM ARCHITECTURE **6**

2.1 Compiler Front-End Pipeline 6

2.2 Project Structure 6

2.3 Technology Stack 7

3 IMPLEMENTATION DETAILS **7**

3.1 Lexical Analyser 7

3.2 Parser — LALR(1) 8

3.2.1 LALR(1) Parsing Algorithm 8

3.2.2 Precedence Declaration 9

3.2.3 Grammar Rules with Semantic Actions 9

3.2.4 Parsing Trace 10

3.3 Abstract Syntax Tree (AST) 10

3.3.1 AST Construction Example — Parse Tree 11

3.4 Evaluation — Truth Value Computation 12

3.5 Translation — Prefix Notation Generation 12

3.6 Graphical User Interface 13

3.6.1 GUI Interaction Flow 14

3.7 Integration of Parser and Translation 14

4 TESTING AND VERIFICATION **15**

4.1 Test Cases 15

4.2 Key Test Cases 15

4.3 Error Handling 16

4.4 Automated Test Suite 16

5 CONCLUSION **17**

REFERENCES **19**

APPENDIX A: SOURCE CODE AVAILABILITY **20**

CHAPTER 1

INTRODUCTION AND LANGUAGE DESIGN

1.1 Background and Problem Statement

Propositional logic is a foundational component of mathematical logic and computer science, dealing with propositions that are either **true** or **false** and the connectives used to combine them [Ben-Ari, 2012]. This report presents a **Propositional Logic Evaluator** that accepts expressions composed of truth values (`t`, `f`) and logical connectives ($\wedge$, $\vee$), and produces two outputs: the **evaluated truth value** and an equivalent expression in **prefix (Polish) notation**. Parenthesised grouping overrides default precedence, and $\wedge$ has higher priority than $\vee$. For example, `t ∨ f ∧ f` evaluates to `t` with prefix notation `(∨ t (∧ f f))`.

The implementation applies compiler front-end techniques [Aho et al., 2006]: lexical analysis, parsing with syntax-directed AST construction, and AST-based evaluation and translation. The GUI is built with PySide6 (Qt6 for Python) and supports both Unicode ($\wedge$, $\vee$) and ASCII (`&`, `|`) operator input. Negation ($\neg$), implication ($\rightarrow$), and biconditional ($\leftrightarrow$) are outside the scope of this assignment.

1.2 Propositional Logic

The language of propositional logic expressions is defined *recursively* (as specified in the assignment):

1. Any of the truth values `t` (true) and `f` (false) is a propositional logic expression.
2. If B and B' are propositional logic expressions, then $B \wedge B'$ and $B \vee B'$ are propositional logic expressions.

The assignment requires expressions built from `t`, `f`, $\wedge$, and $\vee$ with $\wedge$ binding tighter than $\vee$. In addition, this implementation supports parenthesised grouping as an extension: if B is an expression then (B) is an expression, allowing the user to explicitly override default precedence.

Table 1.1. Truth tables for conjunction ($\wedge$) and disjunction ($\vee$)

p	q	$p \wedge q$	$p \vee q$
t	t	t	t
t	f	f	t
f	t	f	t
f	f	f	f

Conjunction ($\wedge$) yields `t` only when both operands are `t`; **disjunction** ($\vee$) yields `t` when at least one operand is `t`.

1.3 Alphabet and Token Specification

The input language is defined over the alphabet shown in Table 1.2. Each symbol or symbol class is associated with a token type produced by the lexical analyser.

Table 1.2. Token specification for the propositional logic language

Token Name	Regular Expression	Matches	Description
`TRUE`	`t`	`t`	Boolean true literal
`FALSE`	`f`	`f`	Boolean false literal
`AND`	`[∧&]`	∧ or `&`	Conjunction operator
`OR`	`[∨|]`	∨ or `|`	Disjunction operator
`LPAREN`	`\(`	`(`	Left parenthesis
`RPAREN`	`\)`	`)`	Right parenthesis

Both Unicode symbols ($\wedge$, $\vee$) and ASCII alternatives (`&`, `|`) are accepted. Whitespace (spaces and tabs) is silently ignored.

1.4 Context-Free Grammar

The syntax of the propositional logic language is defined by the following context-free grammar (CFG):

$$G = (V_N,\ V_T,\ P,\ S) \tag{1.1}$$

where:

- $V_N = \{$ statement, expr $\}$ — non-terminal symbols
- $V_T = \{$ `TRUE`, `FALSE`, `AND`, `OR`, `LPAREN`, `RPAREN` $\}$ — terminal symbols

- S = statement — start symbol
- P — production rules, defined below

The production rules are:

Table 1.3. Production rules of the context-free grammar

Rule	Production	Description
(1)	statement $\rightarrow$ expr	Top-level entry point
(2)	expr $\rightarrow$ expr `OR` expr	Disjunction
(3)	expr $\rightarrow$ expr `AND` expr	Conjunction
(4)	expr $\rightarrow$ `LPAREN` expr `RPAREN`	Parenthesised grouping
(5)	expr $\rightarrow$ `TRUE`	True literal
(6)	expr $\rightarrow$ `FALSE`	False literal

Ambiguity. Rules (2) and (3) make the grammar ambiguous because the expression `t` $\vee$ `f` $\wedge$ `f` admits two distinct parse trees: one where $\vee$ is the root (grouping as $\texttt{t} \vee (\texttt{f} \wedge \texttt{f})$) and one where $\wedge$ is the root (grouping as $(\texttt{t} \vee \texttt{f}) \wedge \texttt{f}$).

1.4.1 Equivalent Unambiguous Grammar

The standard compiler-theory technique for eliminating this ambiguity is *grammar stratification* [Aho et al., 2006]: a separate non-terminal is introduced for each precedence level so that the grammar structure itself enforces the correct grouping. The equivalent unambiguous grammar is:

$$E \rightarrow E \ \vee \ T \ \mid \ T \tag{1.2}$$

$$T \rightarrow T \ \wedge \ F \ \mid \ F \tag{1.3}$$

$$F \rightarrow \texttt{(} \ E \ \texttt{)} \ \mid \ \texttt{t} \ \mid \ \texttt{f} \tag{1.4}$$

Here E handles disjunction (lowest precedence), T handles conjunction (higher precedence), and F represents atomic values and parenthesised sub-expressions. Left recursion in (1.2) and (1.3) encodes left-associativity. Because each operator appears at exactly one level, the grammar admits a unique parse tree for every valid input.

In this implementation, the ambiguous grammar from Table 1.3 is retained and the ambiguity is instead resolved by declaring operator **precedence** and **associativity** in the LALR(1) parser

(Section 1.5). SLY's precedence mechanism achieves the same disambiguation automatically, while keeping the production rules simpler and closer to the original language definition [Beazley, 2023].

1.5 Operator Precedence and Associativity

To disambiguate the grammar, precedence levels and associativity are declared as shown in Table 1.4.

Table 1.4. Operator precedence and associativity

Precedence Level	Operator	Associativity
1 (lower)	$\vee$ (OR)	Left
2 (higher)	$\wedge$ (AND)	Left

Consequence: $\wedge$ binds tighter than $\vee$. In the expression `t ∨ f ∧ f`:

$$\mathtt{t} \vee \mathtt{f} \wedge \mathtt{f} \equiv \mathtt{t} \vee (\mathtt{f} \wedge \mathtt{f}) \neq (\mathtt{t} \vee \mathtt{f}) \wedge \mathtt{f} \tag{1.5}$$

Left associativity means that chains of the same operator group from left to right:

$$\mathtt{t} \vee \mathtt{f} \vee \mathtt{f} \equiv (\mathtt{t} \vee \mathtt{f}) \vee \mathtt{f} \tag{1.6}$$

These declarations are passed to the LALR(1) parser generator, which uses them to resolve shift/reduce conflicts in the generated parse table.

1.6 Notation Systems

The evaluator translates infix expressions into **prefix (Polish) notation** [Łukasiewicz, 1957], where the operator precedes its operands. Table 1.5 compares the three standard notation systems.

Table 1.5. Comparison of notation systems

Notation	Name	Example	Operator Position
Infix	Standard	`t ∧ f`	Between operands
Prefix	Polish	`(∧ t f)`	Before operands
Postfix	Reverse Polish	`t f ∧`	After operands

Prefix notation eliminates the need for precedence rules because the grouping structure is fully explicit through parenthesisation.

CHAPTER 2

SYSTEM ARCHITECTURE

2.1 Compiler Front-End Pipeline

The evaluator follows a classical compiler front-end pipeline [Aho et al., 2006] consisting of three sequential phases: lexical analysis, syntactic analysis, and semantic processing (evaluation and translation). Figure 2.1 illustrates the overall data flow.

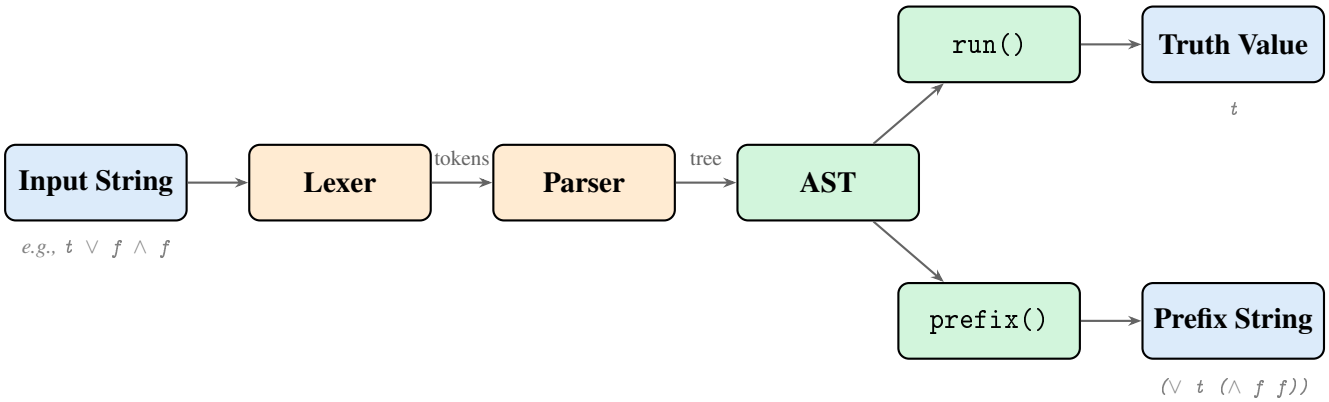


Figure 2.1. System pipeline: input string → lexer → parser → AST → evaluation and translation

Table 2.1 summarises the input, output, and responsible component for each phase.

Table 2.1. Phases of the compiler front-end pipeline

Phase	Input	Output	Component
1. Lexical Analysis	Character string	Token stream	`PropLogicLexer`
2. Syntactic Analysis	Token stream	AST (tree)	`PropLogicParser`
3a. Evaluation	AST	Boolean (`t`/`f`)	`Expression.run()`
3b. Translation	AST	Prefix string	`Expression.prefix()`

2.2 Project Structure

The source code is organised into a modular package structure under `src/125970eval/`:

Table 2.2. Project file structure and component responsibilities

File	Responsibility
`main.py`	Entry point; GUI window and pipeline orchestration
`components/lexica.py`	Lexical analyser (tokeniser)
`components/parsers.py`	LALR(1) parser (AST builder)
`components/ui.py`	PySide6 GUI layout definition
`components/ast/statement.py`	AST node class hierarchy

2.3 Technology Stack

Table 2.3 lists the tools and libraries used in this project.

Table 2.3. Technology stack

Technology	Version	Purpose
Python	≥ 3.10	Core implementation language
SLY [Beazley, 2023]	latest (GitHub)	Lexer and LALR(1) parser generator
PySide6 [The Qt Company, 2024]	$\geq 6.10.2$	Qt6-based GUI framework
uv [Astral, 2024]	latest	Python package and virtual environment manager

CHAPTER 3

IMPLEMENTATION DETAILS

3.1 Lexical Analyser

The lexical analyser is implemented in `components/lexica.py` as the class `PropLogicLexer`, extending `sly.Lexer`. It converts an input string into a stream of tokens using regular expression pattern matching.

```
class PropLogicLexer(Lexer):
    tokens = { TRUE, FALSE, AND, OR, LPAREN, RPAREN }

    ignore = ' \t'

    AND    = r'[∧&]'
    OR     = r'[∨|]'
    LPAREN = r'\('
    RPAREN = r'\)'
    TRUE   = r't'
    FALSE  = r'f'

    def error(self, t):
```

```
        bad = t.value[0]
        self.index += 1
        raise ValueError(
            f"Illegal character '{bad}' "
            f"at position {self.index}")
```

Listing 3.1: Token definitions in `PropLogicLexer`

Key design choices:

- Both Unicode (∧, ∨) and ASCII (`&`, `|`) operators are accepted via character class regex patterns.
- The `ignore` attribute silently consumes whitespace.
- The `error()` method raises a `ValueError` with a message identifying the illegal character and its position. This exception propagates up to the GUI, which displays the error to the user.

Tokenisation example. For the input `t ∨ f ∧ f`, the lexer produces the token stream shown in Table 3.1.

Table 3.1. Tokenisation of `t ∨ f ∧ f`

Position	Character	Token Type	Token Value
0	t	TRUE	t
2	∨	OR	∨
4	f	FALSE	f
6	∧	AND	∧
8	f	FALSE	f

Whitespace at positions 1, 3, 5, and 7 is consumed by the `ignore` directive.

3.2 Parser — LALR(1)

The parser is implemented in `components/parsers.py` as the class `PropLogicParser`, extending `sly.Parser`. It uses the **LALR(1)** (Look-Ahead Left-to-right Rightmost-derivation) parsing algorithm [Aho et al., 2006].

3.2.1 LALR(1) Parsing Algorithm

LALR(1) is a **bottom-up** parsing algorithm that reads input left to right, produces a rightmost derivation in reverse, and uses one token of lookahead [Aho et al., 2006]. It operates via a **shift-**

reduce mechanism: *shift* pushes the next token onto the parse stack, while *reduce* replaces a sequence matching a production's right-hand side with the corresponding non-terminal, executing the associated semantic action (AST node construction). Parsing runs in $O(n)$ time with respect to input length, and the SLY library automatically generates the LALR(1) parse table from the grammar rules and precedence declarations.

3.2.2 Precedence Declaration

```
precedence = (
    ('left', OR),       # Level 1 - lower priority
    ('left', AND),      # Level 2 - higher priority
)
```

Listing 3.2: Precedence declaration in `PropLogicParser`

In SLY, tokens listed **later** in the precedence tuple bind **tighter**. This ensures that $\wedge$ has higher precedence than $\vee$. When a shift/reduce conflict arises, the parser compares the precedence of the lookahead token with the precedence of the reduction rule and chooses accordingly.

3.2.3 Grammar Rules with Semantic Actions

Each grammar rule is implemented as a decorated method that constructs the corresponding AST node:

```
@_('expr OR expr')
def expr(self, p) -> Expression:
    return Expression_logic(Operations.OR, p.expr0, p.expr1)

@_('expr AND expr')
def expr(self, p) -> Expression:
    return Expression_logic(Operations.AND, p.expr0, p.expr1)

@_('LPAREN expr RPAREN')
def expr(self, p) -> Expression:
    return p.expr

@_('TRUE')
def expr(self, p) -> Expression:
    return Expression_bool(True)

@_('FALSE')
def expr(self, p) -> Expression:
    return Expression_bool(False)
```

Listing 3.3: Grammar rules with semantic actions

The semantic actions are **syntax-directed**: each grammar rule directly constructs the corre-

sponding AST node, building the tree bottom-up as reductions occur during parsing.

3.2.4 Parsing Trace

Table 3.2 shows the step-by-step LALR(1) shift-reduce parsing process for the input `t ∨ f ∧ f`, demonstrating how precedence resolves the ambiguity.

Table 3.2. LALR(1) shift-reduce parsing trace for `t ∨ f ∧ f`

Step	Stack	Input	Action	Semantic Action
1	—	`t ∨ f ∧ f`	Shift `t`	—
2	`TRUE`	`∨ f ∧ f`	Reduce (5)	`Expression_bool(True)`
3	`expr`	`∨ f ∧ f`	Shift ∨	—
4	`expr ∨`	`f ∧ f`	Shift `f`	—
5	`expr ∨ FALSE`	`∧ f`	Reduce (6)	`Expression_bool(False)`
6	`expr ∨ expr`	`∧ f`	Shift ∧ *	—
7	`expr ∨ expr ∧`	`f`	Shift `f`	—
8	`expr ∨ expr ∧ FALSE`	—	Reduce (6)	`Expression_bool(False)`
9	`expr ∨ expr ∧ expr`	—	Reduce (3)	`Expression_logic(AND, ...)`
10	`expr ∨ expr`	—	Reduce (2)	`Expression_logic(OR, ...)`
11	`statement`	—	Accept	Return root AST node

* At step 6, a **shift/reduce conflict** occurs: the parser could either reduce `expr ∨ expr` by rule (2) or shift ∧. Because AND has **higher precedence** than OR, the parser chooses to **shift**, correctly binding ∧ tighter.

3.3 Abstract Syntax Tree (AST)

The AST is implemented in `components/ast/statement.py` using an inheritance hierarchy of three classes. Figure 3.1 shows the class structure.

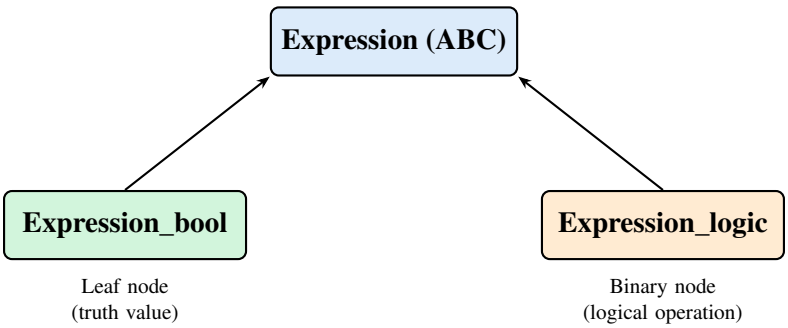


Figure 3.1. AST class hierarchy

Table 3.3 summarises each class.

Table 3.3. AST node classes

Class	Role	Fields	Key Methods
`Expression`	Abstract base class	`value`	`run()`, `prefix()` (abstract)
`Expression_bool`	Leaf: truth literal	`value: bool`	`run()` → `bool`; `prefix()` → `"t"` or `"f"`
`Expression_logic`	Binary: logical op	`operation`, `left`, `right`	`run()` → recursive eval; `prefix()` → prefix string

The `Operations` enum defines the two operations:

```
class Operations(Enum):
    AND = 0
    OR  = 1
```

Listing 3.4: `Operations` enum

3.3.1 AST Construction Example — Parse Tree

For the input expression `t ∨ f ∧ f`, the parser constructs the AST shown in Figure 3.2. This tree is the direct result of the LALR(1) parsing process described in Table 3.2.

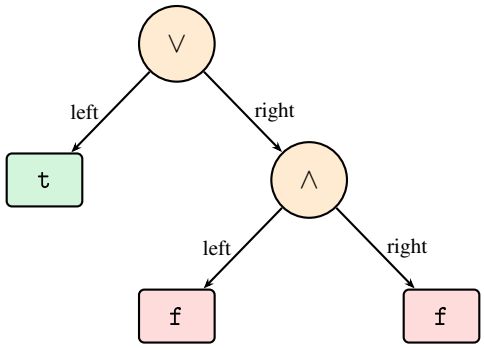


Figure 3.2. AST (parse tree) for `t ∨ f ∧ f` — ∧ binds tighter and forms a subtree under ∨

Figure 3.3 shows the AST for the parenthesised expression `(t ∨ f) ∧ f`, where parentheses override the default precedence.

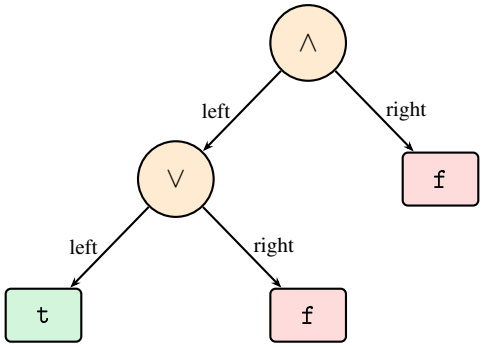


Figure 3.3. AST (parse tree) for `(t ∨ f) ∧ f` — parentheses force ∨ to be evaluated first

3.4 Evaluation — Truth Value Computation

The `run()` method evaluates the AST by recursive **post-order traversal**. Leaf nodes return their boolean value directly; binary nodes recursively evaluate both children and then apply the logical operation.

$$\texttt{run}() = \begin{cases} \texttt{value} & \text{(leaf node)} \\ \texttt{left.run()} \wedge \texttt{right.run()} & \text{if operation} = \text{AND} \\ \texttt{left.run()} \vee \texttt{right.run()} & \text{if operation} = \text{OR} \end{cases} \tag{3.1}$$

```python
def run(self):
    left_val = self.left.run()
    right_val = self.right.run()
    if self.operation == Operations.AND:
        self.value = left_val and right_val
    elif self.operation == Operations.OR:
        self.value = left_val or right_val
    return self.value
```

Listing 3.5: Evaluation method in `Expression_logic`

3.5 Translation — Prefix Notation Generation

The `prefix()` method converts the AST into a fully parenthesised prefix notation string via recursive **pre-order traversal**. Leaf nodes return `"t"` or `"f"`; binary nodes output `(op left right)`.

$$\texttt{prefix}() = \begin{cases} \texttt{"t"}/\texttt{"f"} & \text{(leaf node)} \\ \texttt{"("} + op + \texttt{" "} + \texttt{left.prefix()} + \texttt{" "} + \texttt{right.prefix()} + \texttt{")"} & \text{(binary node)} \end{cases} \tag{3.2}$$

```python
def prefix(self) -> str:
    op_str = '∧' if self.operation == Operations.AND else '∨'
    return f"({op_str} {self.left.prefix()} {self.right.prefix()})"
```

Listing 3.6: Prefix translation in `Expression_logic`

The output is fully parenthesised, making grouping explicit without relying on precedence rules.

3.6 Graphical User Interface

The GUI is implemented using **PySide6** (Qt6 for Python) across two files: `components/ui.py` defines the layout and `main.py` implements the event-handling logic.

Figure 3.4 shows the GUI layout:

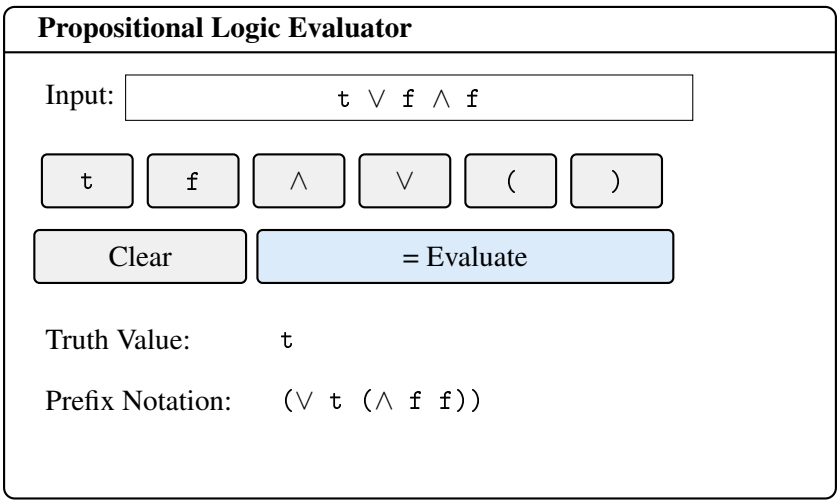
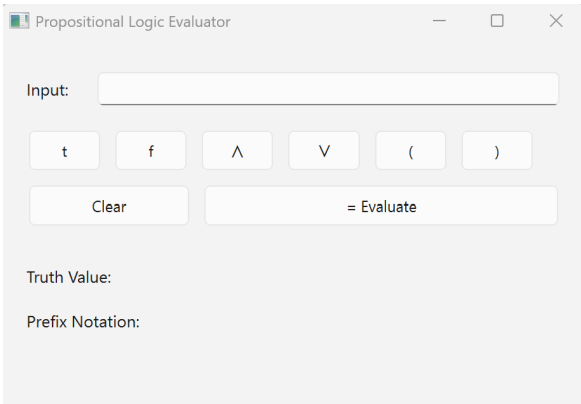
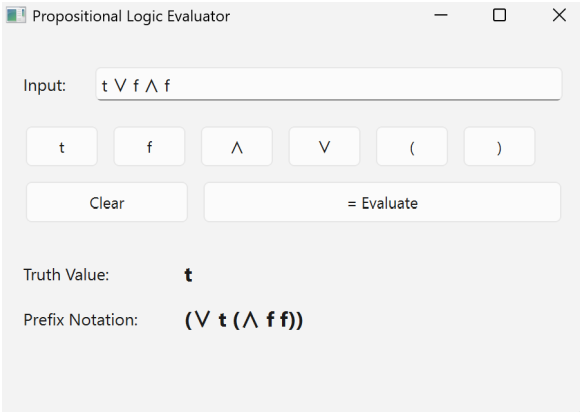


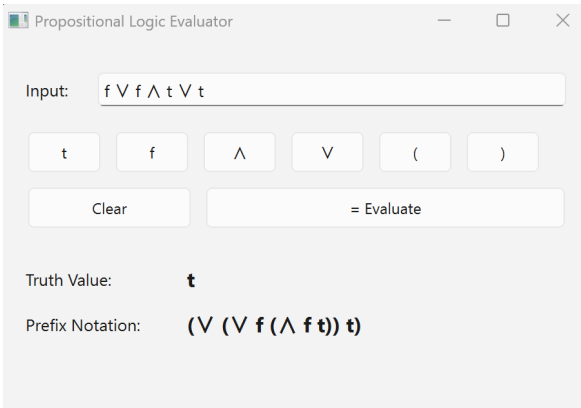
Figure 3.4. GUI layout of the Propositional Logic Evaluator



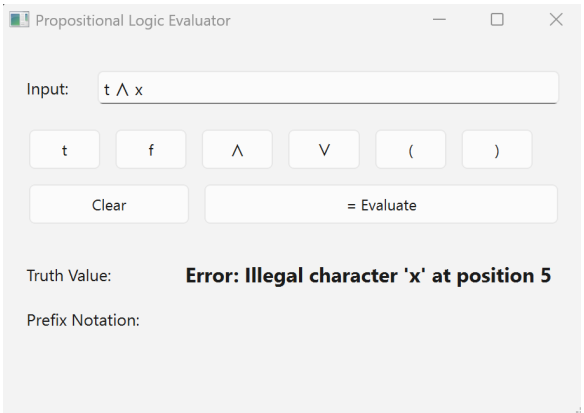
(a) Empty GUI on launch



(b) Operator precedence: `t ∨ f ∧ f ⇒ t`



(c) Left-associativity: `f ∨ f ∧ t ∨ t ⇒ t`



(d) Error handling: illegal character `x`

Figure 3.5. Application screenshots — GUI layout, operator precedence, left-associativity, and error handling

3.6.1 GUI Interaction Flow

When the user clicks **Evaluate**, a fresh lexer and parser are instantiated. The input text is tokenised, parsed into an AST, and the AST's `run()` and `prefix()` methods produce the truth value and prefix string, respectively. If the input is invalid, an error message is displayed instead.

3.7 Integration of Parser and Translation

The assignment requires both a truth value and a prefix notation string from a single input. Rather than embedding computation inside grammar rules, this implementation uses the AST as an **intermediate representation** that decouples parsing from downstream processing:

1. **Parser → AST:** Each grammar rule's semantic action (Listing 3.3) constructs an AST node during bottom-up reduction — a form of *syntax-directed translation* [Aho et al.,

2006].

2. **AST → Truth value:** `run()` performs a post-order traversal (Listing 3.5).

3. **AST → Prefix string:** `prefix()` performs a pre-order traversal (Listing 3.6).

Because both traversals operate on the *same* tree, the expression is parsed only once:

```
result = parser.parse(lexer.tokenize(input_text))  # parse once
truth_value = result.run()                          # traversal 1
prefix_str  = result.prefix()                       # traversal 2
```

Listing 3.7: Pipeline integration in `MainWindow.evaluate()`

This clean separation means adding a new output format (e.g. postfix notation) requires only a new traversal method — the parser and grammar remain unchanged.

CHAPTER 4

TESTING AND VERIFICATION

4.1 Test Cases

All six required test cases were executed and verified against expected outputs. Table 4.1 presents the complete results.

Table 4.1. Test case results — all cases pass

#	Input	Expected Value	Actual Value	Expected Prefix	Actual Prefix
1	t	t	t	t	t
2	f	f	f	f	f
3	t ∧ f	f	f	(∧ t f)	(∧ t f)
4	t ∨ f	t	t	(∨ t f)	(∨ t f)
5	t ∨ f ∧ f	t	t	(∨ t (∧ f f))	(∨ t (∧ f f))
6	(t ∨ f) ∧ f	f	f	(∧ (∨ t f) f)	(∧ (∨ t f) f)

Result: All 6 test cases pass. Expected and actual outputs match for both truth value and prefix notation.

4.2 Key Test Cases

Test 5 (`t ∨ f ∧ f`) validates operator precedence: with $\wedge > \vee$, the expression is parsed as $\mathtt{t} \vee (\mathtt{f} \wedge \mathtt{f}) = \mathtt{t}$ (not $(\mathtt{t} \vee \mathtt{f}) \wedge \mathtt{f} = \mathtt{f}$). The prefix output `(∨ t (∧ f f))` confirms correct

grouping (see Figure 3.2).

Test 6 (`(t ∨ f) ∧ f`) validates that parentheses override default precedence, yielding $(\texttt{t} \vee \texttt{f}) \wedge \texttt{f} = \texttt{f}$ with prefix `(∧ (∨ t f) f)` (see Figure 3.3).

4.3 Error Handling

The evaluator handles invalid input at three levels:

- **Lexical errors:** If the input contains an unrecognised character (e.g. `x`), the lexer's `error()` method raises a `ValueError` with a descriptive message including the offending character and its position. This exception propagates through the pipeline and is caught by the GUI's `evaluate()` method, which displays the error message directly in the truth-value output field. The user therefore sees, for example, "`Error: Illegal character 'x' at position 5`" instead of a silent failure.
- **Syntax errors:** If the token stream does not conform to the grammar (e.g. `t t` or `t ∧`), no valid AST is returned (the parser yields `None` in this implementation), and the GUI displays an error message instead of a result.
- **Runtime exceptions:** Any unexpected exception during the pipeline is caught by the `evaluate()` method, and the error message is displayed in the truth value output field.

This layered approach ensures the application never crashes on malformed input and always provides meaningful feedback to the user.

4.4 Automated Test Suite

The project includes an automated test suite (`tests/test_evaluator.py`) executed with **pytest**. The suite contains 12 tests organised into three groups:

Table 4.2. Automated test suite summary

Test Group	Count	Purpose
`TestCoreCases`	6	The six representative test cases from Table 4.1: literals, conjunction, disjunction, operator precedence, and parenthesised grouping
`TestASCIIOperators`	3	Verifies that the ASCII alternatives (`&`, `|`) produce identical results to the Unicode operators ($\wedge$, $\vee$)
`TestErrorHandling`	3	Checks that illegal characters raise `ValueError`, incomplete expressions return `None`, and empty input returns `None`
Total	**12**	**All pass**

Running the suite:

```
uv run pytest tests/test_evaluator.py -v
```

CHAPTER 5

CONCLUSION

This report presented a **Propositional Logic Evaluator** that applies classical compiler front-end techniques to evaluate expressions over truth values (`t`, `f`) with conjunction ($\wedge$) and disjunction ($\vee$). The system follows a three-phase pipeline — lexical analysis, LALR(1) parsing with syntax-directed AST construction (via SLY [Beazley, 2023]), and AST-based evaluation and translation — producing both the evaluated truth value and an equivalent prefix notation string from a single parse [Aho et al., 2006]. An interactive PySide6 GUI with dual Unicode/ASCII input modes completes the implementation. All six required test cases pass correctly (Chapter 4, Table 4.1).

Three design choices merit emphasis: using an **LALR(1) parser generator** keeps the grammar declarative and aligned with formal BNF notation; constructing an **AST as an intermediate representation** decouples parsing from downstream processing, enabling independent tree traversals for evaluation and translation without re-parsing; and producing **fully parenthesised prefix notation** ensures unambiguous output regardless of operator precedence. Future

extensions could include additional connectives ($\neg$, $\rightarrow$, $\leftrightarrow$), named propositional variables, automatic truth table generation, and step-by-step evaluation visualisation.

REFERENCES

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.

Astral. uv — an extremely fast python package installer and resolver. `https://github.com/astral-sh/uv`, 2024. Accessed: January 2026.

David Beazley. SLY — sly lex yacc. `https://github.com/dabeaz/sly`, 2023. Accessed: January 2026.

Mordechai Ben-Ari. *Mathematical Logic for Computer Science*. Springer, London, 3rd edition, 2012.

Jan Łukasiewicz. *Aristotle's Syllogistic from the Standpoint of Modern Formal Logic*. Oxford University Press, Oxford, 2nd edition, 1957.

The Qt Company. Qt for Python — PySide6 documentation. `https://doc.qt.io/qtforpython-6/`, 2024. Accessed: January 2026.

APPENDIX A: SOURCE CODE AVAILABILITY

The complete source code for the Propositional Logic Evaluator is publicly available on GitHub:

`https://github.com/limhpone/`
`125970-Assignment-1-Propositional-Logic-Evaluator-PLC.git`

A video walkthrough demonstrating the evaluator in action is available on YouTube:

`https://youtu.be/jVUvnhstTXY`

The repository contains the following components:

- **Lexical analyser** (`components/lexica.py`) — tokenises input expressions into `TRUE`, `FALSE`, `AND`, `OR`, `LPAREN`, `RPAREN`
- **LALR(1) parser** (`components/parsers.py`) — parses token streams and builds an abstract syntax tree
- **AST node classes** (`components/ast/statement.py`) — `Expression_bool` (leaf) and `Expression_logic` (binary) with `run()` and `prefix()` methods
- **GUI layout** (`components/ui.py`) — PySide6 interface definition
- **Entry point** (`main.py`) — GUI window creation and pipeline orchestration

How to Run

1. Clone the repository:

```
git clone https://github.com/limhpone/
    125970-Assignment-1-Propositional-Logic-Evaluator-PLC.git
```

2. Install dependencies using `uv`:

```
uv sync
```

3. Run the application:

```
cd src/125970eval
uv run main.py
```

Source Code Listings

The key implementation files are reproduced below for reference.

A.1 Lexical Analyser — `components/lexica.py`

```python
from sly import Lexer

class PropLogicLexer(Lexer):
    tokens = { TRUE, FALSE, AND, OR, LPAREN, RPAREN }

    ignore = ' \t'

    AND    = r'[∧&]'
    OR     = r'[∨|]'
    LPAREN = r'\('
    RPAREN = r'\)'
    TRUE   = r't'
    FALSE  = r'f'

    @_(r'\n+')
    def ignore_newline(self, t):
        self.lineno += t.value.count('\n')

    def error(self, t):
        bad = t.value[0]
        self.index += 1
        raise ValueError(
            f"Illegal character '{bad}' "
            f"at position {self.index}")
```

A.2 Parser — `components/parsers.py`

```python
from components.lexica import PropLogicLexer
from components.ast.statement import (Expression, Expression_bool,
                                      Expression_logic, Operations)
from sly import Parser

```

```python
class PropLogicParser(Parser):
    debugfile = 'parser.out'
    start = 'statement'
    tokens = PropLogicLexer.tokens
    precedence = (
        ('left', OR),
        ('left', AND),
    )

    @_('expr')
    def statement(self, p):
        return p.expr

    @_('expr OR expr')
    def expr(self, p) -> Expression:
        return Expression_logic(Operations.OR, p.expr0, p.expr1)

    @_('expr AND expr')
    def expr(self, p) -> Expression:
        return Expression_logic(Operations.AND, p.expr0, p.expr1)

    @_('LPAREN expr RPAREN')
    def expr(self, p) -> Expression:
        return p.expr

    @_('TRUE')
    def expr(self, p) -> Expression:
        return Expression_bool(True)

    @_('FALSE')
    def expr(self, p) -> Expression:
        return Expression_bool(False)
```

A.3 AST Node Classes — `components/ast/statement.py`

```python
from enum import Enum
from abc import ABC, abstractmethod

class Operations(Enum):
    AND = 0
    OR = 1

class Expression(ABC):
    @abstractmethod
    def __init__(self) -> None:
        self.value = None

    @abstractmethod
    def run(self):
        pass

    @abstractmethod
```

```python
    def prefix(self) -> str:
        pass

class Expression_bool(Expression):
    def __init__(self, value: bool) -> None:
        self.value: bool = value

    def run(self):
        return self.value

    def prefix(self) -> str:
        return 't' if self.value else 'f'

class Expression_logic(Expression):
    def __init__(self, operation: Operations,
                 left: Expression,
                 right: Expression) -> None:
        self.operation = operation
        self.left = left
        self.right = right
        self.value = None

    def run(self):
        left_val = self.left.run()
        right_val = self.right.run()
        if self.operation == Operations.AND:
            self.value = left_val and right_val
        elif self.operation == Operations.OR:
            self.value = left_val or right_val
        return self.value

    def prefix(self) -> str:
        op_str = '∧' if self.operation == Operations.AND else '∨'
        return (f"({op_str} {self.left.prefix()} "
                f"{self.right.prefix()})")
```

A.4 Main Entry Point — main.py

```python
import sys
from PySide6.QtWidgets import QApplication, QMainWindow
from components.lexica import PropLogicLexer
from components.parsers import PropLogicParser
from components.ui import Ui_MainWindow

class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)

        self.ui.button_t.clicked.connect(lambda: self.push("t"))
        self.ui.button_f.clicked.connect(lambda: self.push("f"))
```

```
        self.ui.button_and.clicked.connect(
            lambda: self.push(" \u2227 "))
        self.ui.button_or.clicked.connect(
            lambda: self.push(" \u2228 "))
        self.ui.button_lparen.clicked.connect(
            lambda: self.push("("))
        self.ui.button_rparen.clicked.connect(
            lambda: self.push(")"))
        self.ui.button_clear.clicked.connect(self.clear)
        self.ui.button_equal.clicked.connect(self.evaluate)

    def push(self, text: str):
        current_text = self.ui.input_text.text()
        self.ui.input_text.setText(f"{current_text}{text}")

    def clear(self):
        self.ui.input_text.setText("")
        self.ui.output_value.setText("")
        self.ui.output_prefix.setText("")

    def evaluate(self):
        lexer = PropLogicLexer()
        parser = PropLogicParser()
        input_text = self.ui.input_text.text()
        try:
            result = parser.parse(lexer.tokenize(input_text))
            if result is not None:
                truth_value = result.run()
                self.ui.output_value.setText(
                    't' if truth_value else 'f')
                self.ui.output_prefix.setText(result.prefix())
            else:
                self.ui.output_value.setText(
                    "Error: Invalid expression")
                self.ui.output_prefix.setText("")
        except ValueError as e:
            self.ui.output_value.setText(f"Error: {e}")
            self.ui.output_prefix.setText("")
        except Exception as e:
            self.ui.output_value.setText(f"Error: {e}")
            self.ui.output_prefix.setText("")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
```